



LINFO1121 2025-2026

Algorithmique et Structure de données

Complexité, Type-Abstrait de données, Structure Chainées

TA LLN: Harold Kiossou, Alice Burlats, Achille Morenville (Competition) + Tutors

TA Charleroi: Roger Kameugne

Prof pierre.schaus@uclouvain.be

Plan

- Motivation
- Organisation du cours
- Rappel Java
- Type Abstrait de données
- Complexité
- Les outils
 - Java util collection
 - IDE IntelliJ
 - Junit

Why study algorithms ?

Because it is the foundation of computer science.

The skills acquired in LINFO1121 are used in many other courses.

Did you know that Dijkstra's algorithm is used in every Internet router?

The role of algorithms in our society is growing. People speak of an "algorithmic society."

If you do not master the basics, you will not pass their interviews:



Abstract Data Types

- In this course, we manipulate and process data.
- Data can be added and processed via an API (interface or set of methods + constructor).
- The user does not need to know the concrete representation of data storage (they interact with a black box).
- The goal of this course is to study the implementation of these black boxes, because not all implementations are created equal.



```
public static void main(String[] args) {  
    Stack<Integer> stack = null;  
    stack.push(2);  
    stack.push(3);  
    stack.pop();  
    stack.isEmpty()  
}
```

StackImplem

API

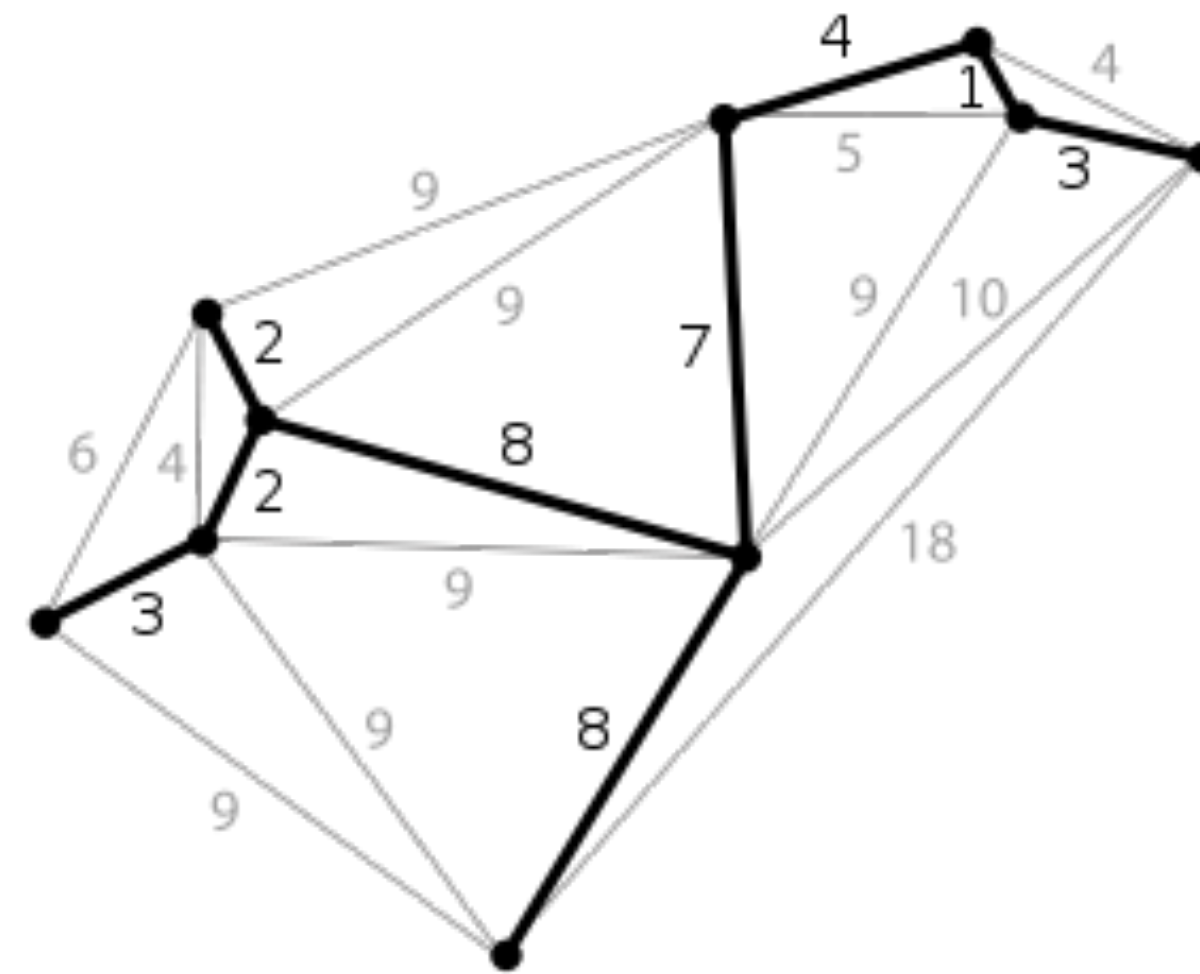
```
public interface Stack<Item> {  
    void push(Item item);  
    Item pop();  
    boolean isEmpty();  
    int size();  
}
```

Concretely, in this course we will study

- Abstract data types to:
 - Insert ordered objects in an arbitrary order and quickly retrieve or remove the maximum
 - Insert/remove objects in an arbitrary order based on an ordered key, and then quickly iterate in order, find the minimum, or retrieve the object associated with any key or its successors
 - Represent networks: add nodes, find neighboring nodes of a given node, etc.
 - Insert and remove objects in an arbitrary order based on a key that is not necessarily ordered
- Algorithms to:
 - Sort data
 - Compress text data
 - Efficiently find substrings within a large string
 - Compute paths or trees in networks

Example: Network Design

- Find a subgraph in this network (graph) that has the smallest possible weight and connects all the nodes.



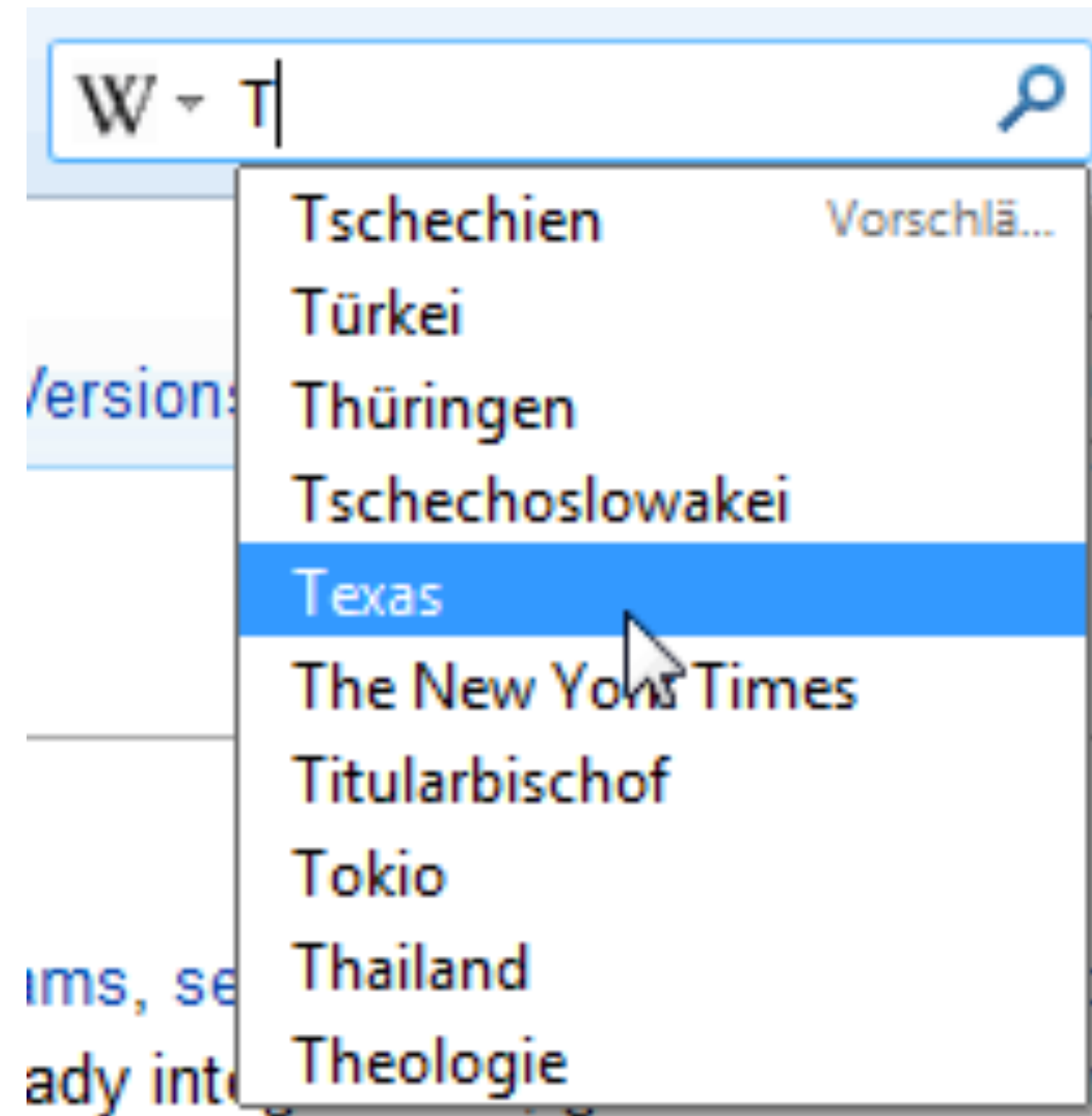
- Application: connect homes at the lowest possible cost using fiber optics

Example: Search Engine

- Search engine: You want to be able to store millions of keywords, and for each one, a list of URLs pointing to the corresponding pages.
- How do you ensure that you can quickly return a URL for a given keyword?

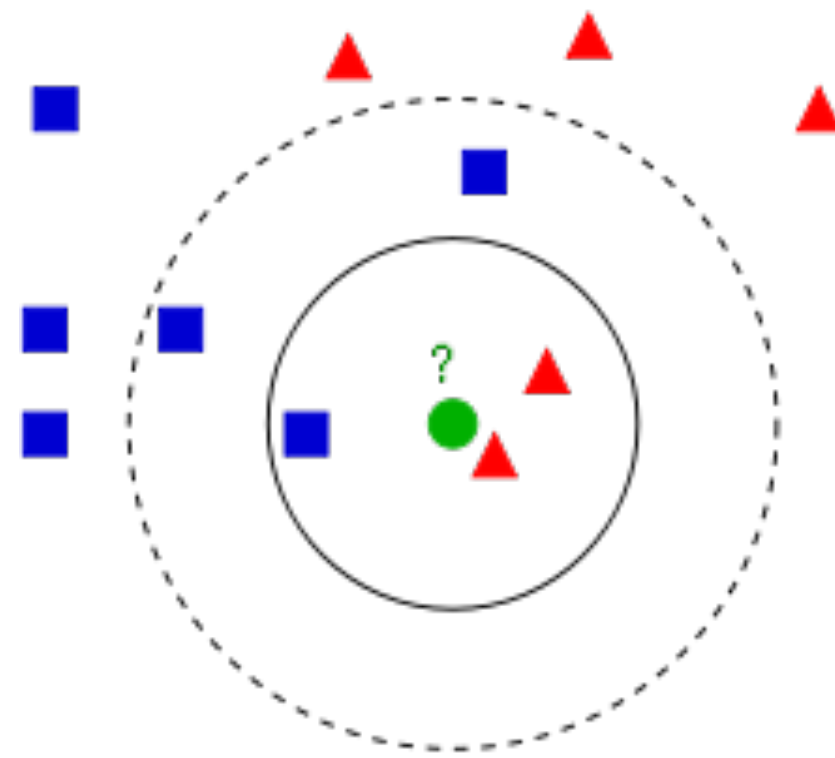


Example: Auto-Complete



Example: Machine Learning

- Is the new point a blue point or a red point?
- Can we count the "k" nearest neighbors and assign the majority class?
- How do we efficiently count the "k" nearest neighbors?



java.util.collection

- <https://docs.oracle.com/javase/8/docs/api/index.html?java/util/Collection.html>

java.rmi.registry
java.rmi.server
java.security
java.security.acl
java.security.cert
java.security.interfaces
java.security.spec
java.sql
java.text
java.text.spi
java.util
java.util.concurrent
java.util.concurrent.atomic
java.util.concurrent.locks
java.util.jar
java.util.logging
java.util.prefs
java.util.regex
java.util.spi
java.util.zip
javax.accessibility

java.util

Interfaces

Collection
Comparator
Deque
Enumeration
EventListener
Formattable
Iterator
List
ListIterator
Map
Map.Entry
NavigableMap
NavigableSet
Observer
Queue
RandomAccess
Set
SortedMap
SortedSet

Classes

AbstractCollection
AbstractList
AbstractMap
AbstractMap.SimpleEntry
AbstractMap.SimpleImmutableEntry
AbstractQueue
AbstractSequentialList
AbstractSet
ArrayDeque
ArrayList
Arrays
BitSet
Calendar
Collections
Currency
Date
Dictionary
EnumMap
EnumSet
EventListenerProxy
EventObject
FormattableFlags

OverviewPackageClassUseTreeDeprecatedIndexHelp

Prev PackageNext PackageFramesNo Frames

Package java.util

Contains the collections framework, legacy collection classes, event model, date and time facilities, internationalization, and miscellaneous utility classes (a string tokenizer, a random-number generator, and a bit array).

See: Description

Interface Summary

Interface	Description
Collection<E>	The root interface in the <i>collection hierarchy</i> .
Comparator<T>	A comparison function, which imposes a <i>total ordering</i> on some collection of objects.
Deque<E>	A linear collection that supports element insertion and removal at both ends.
Enumeration<E>	An object that implements the Enumeration interface generates a series of elements, one at a time.
EventListener	A tagging interface that all event listener interfaces must extend.
Formattable	The Formattable interface must be implemented by any class that needs to perform custom formatting using the 's' conversion specifier of Formatter .
Iterator<E>	An iterator over a collection.
List<E>	An ordered collection (also known as a <i>sequence</i>).
ListIterator<E>	An iterator for lists that allows the programmer to traverse the list in either direction, modify the list during iteration, and obtain the iterator's current position in the list.
Map<K,V>	An object that maps keys to values.
Map.Entry<K,V>	A map entry (key-value pair).
NavigableMap<K,V>	A SortedMap extended with navigation methods returning the closest matches for given search targets.
NavigableSet<E>	A SortedSet extended with navigation methods reporting closest matches for given search targets.
Observer	A class can implement the Observer interface when it wants to be informed of changes in observable objects.
Queue<E>	A collection designed for holding elements prior to processing.
RandomAccess	Marker interface used by List implementations to indicate that they support fast (generally constant time) random access.
Set<E>	A collection that contains no duplicate elements.
SortedMap<K,V>	A Map that further provides a <i>total ordering</i> on its keys.
SortedSet<E>	A Set that further provides a <i>total ordering</i> on its elements.

Class Summary

Class	Description
AbstractCollection<E>	This class provides a skeletal implementation of the Collection interface, to minimize the effort required to implement this interface.
AbstractList<E>	This class provides a skeletal implementation of the List interface to minimize the effort required to implement this interface backed by a "random access" data store (such as an array).
AbstractMap<K,V>	This class provides a skeletal implementation of the Map interface, to minimize the effort required to implement this interface.
AbstractMap.SimpleEntry<K,V>	An Entry maintaining a key and a value.
AbstractMap.SimpleImmutableEntry<K,V>	An Entry maintaining an immutable key and value.
AbstractQueue<E>	This class provides skeletal implementations of some Queue operations.
AbstractSequentialList<E>	This class provides a skeletal implementation of the List interface to minimize the effort required to implement this interface backed by a "sequential access" data store (such as a linked list).
AbstractSet<E>	This class provides a skeletal implementation of the Set interface to minimize the effort required to implement this interface.
ArrayDeque<E>	Resizable-array implementation of the Deque interface.
ArrayList<E>	Resizable-array implementation of the List interface.

- A Java package that contains most abstract data types. This package will hold no more secrets for you by the end of this course.

Pedagogy

- You are at the center of your learning.
- We are only here to guide you and provide activities to help you master the material to get ready for the final exam.



Reference Book



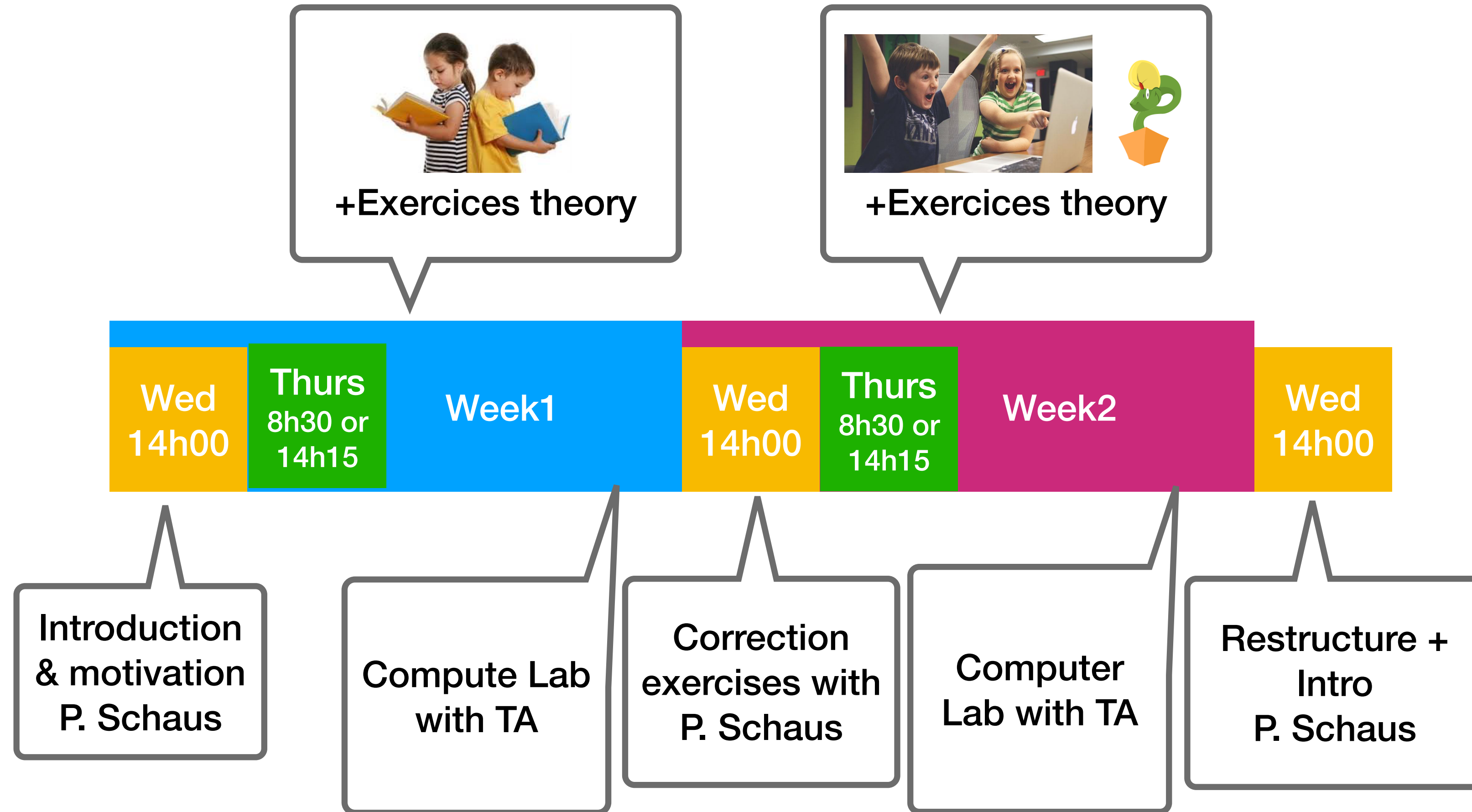
- +-65 euros sur Amazon.BE (paper) (13 sept 2026)

Le site du cours

- <https://moodle.uclouvain.be/course/view.php?id=1049> for announcement and communication
- <https://pschaus.github.io/LINFO1121/> for the rest

Organisation UCLouvain

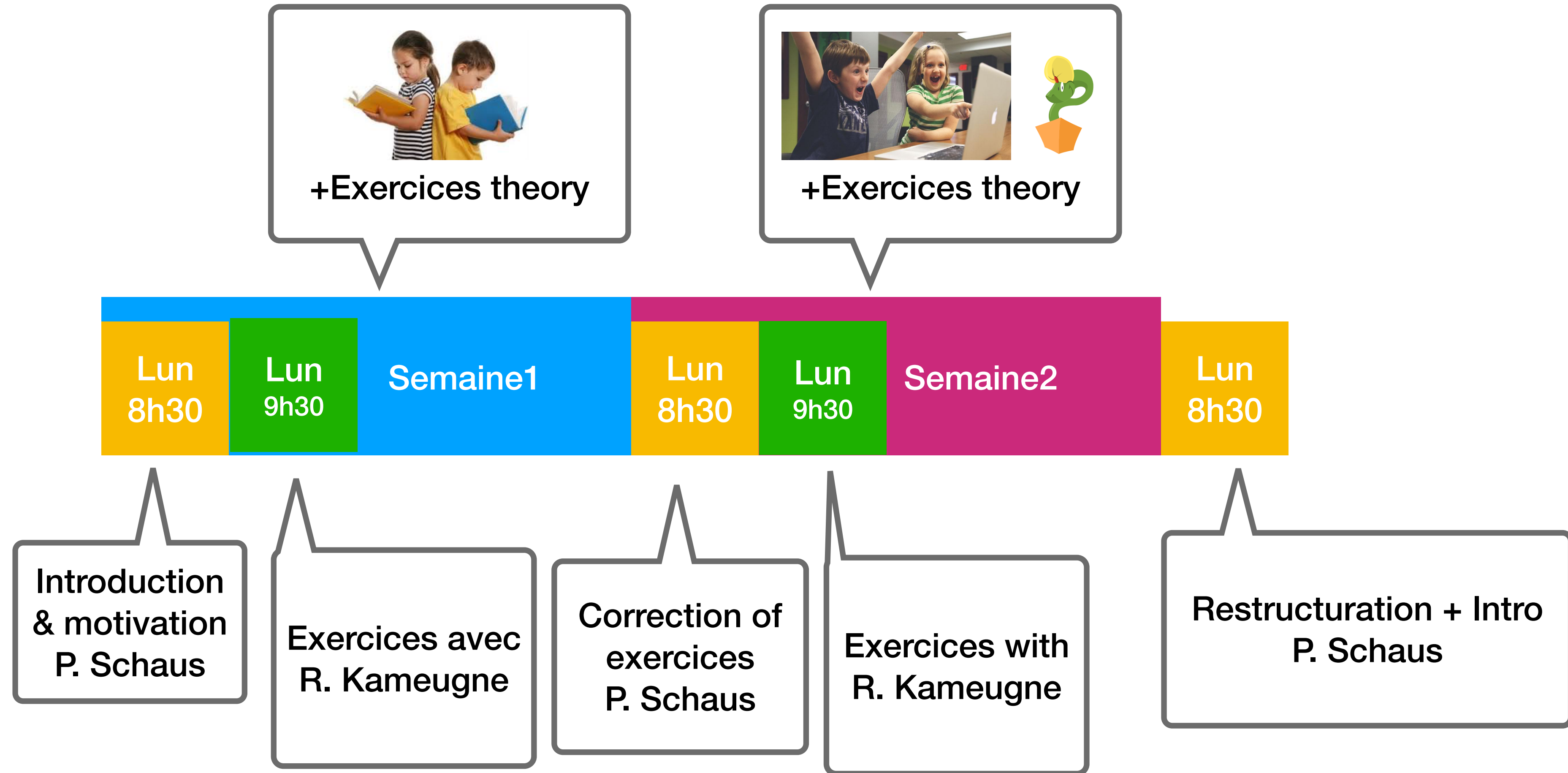
- 6 x 2 Weeks Module



Resster for the lab on Moodle

<https://moodle.uclouvain.be/mod/choicegroup/view.php?id=390918>.

Organisation Charleroi



Evaluation

- Typical Examen
 - 3 (implementation exercises on ingenious) x 20/6 points
 - Maybe a more theoretical question



Programming Contest, 16th of December 2026 13h-16

- LLN, groupes de 2 ou 3



- You get 1 participation point if you (your team) has solved at least two exercises.
- **!!! Contest is AI free !!!**

How is graded the exam

Rough estimation, not a rule

- 50% of the grade for correctness (it computes what it is supposed to compute)
 - 50% if the algorithm further has the right time complexity
- Example: Write the most possible efficient algorithm for sorting numbers.
 - Code correct but in $O(n^2)$: 4/8
 - Code buggy but in $O(n \log(n))$: 0/8
 - Code correct and in $O(n \log(n))$: 8/8

Why Java ?

- Industry standard & popularity: One of the most widely used programming languages in the world.
- Maturity & governance: Created in 1995 at Sun Microsystems; maintained by Oracle since 2009.
- Performance & portability: High-performance, cross-platform execution via the Java Virtual Machine (JVM).
- Course baseline:
 - We will use Java 8 (consistent with the course textbook).
- Prerequisites:
 - Prior familiarity with Java is assumed.
 - * Need a refresher? Review Section 1.1 of the textbook:
 - * Primitive types, Arrays, Instance variables / Object-oriented basics



Why not Python ?

Aspect	Python	Java
Abstraction	Too high-level: Hides memory allocation and Too high-level: Hides memory allocation and fixed-size mechanics.-size mechanics.	Explicit: Direct visibility into fixed-size arrays, references, and memory layouts.
Implementation	Built-ins are implemented in C, not Python (e.g., CPython source).	Standard collections are written in Java—you can read the source directly.
Performance	High interpreter overhead; algorithmic efficiency is masked by C-extensions.	Fast, predictable JIT execution that mirrors true asymptotic complexity.

Ininious, Inginious, Inginious and IntelliJ

- ⚠ Very hard course ! ⚠ (some students pass it 6 times)
- Don't be one of them. The only way to succeed is to practice regularly (without AI).
- Spend time at debugging your program and do all the exercises we propose (many of them are previous exams). Master IntelliJ (don't discover it the day of the exam)



"Patience you must have my young Padawan."



Generative AI

- Why Skip AI for Exam Prep?
- The Exam is Unassisted: You will have IntelliJ IDE, paper, and your brain—no AI, prompts, or (basic autocomplete offered by IntelliJ).
- The Illusion of Competence: Reading AI code is not the same as deriving algorithms from scratch under time pressure; **real intuition requires personal struggle**.
- It is perfectly normal to **spend hours on a single exercise**, trying to debug it, using breakpoints and println. Be patient, this is the only way to prepare for the exam. Debugging and testing skills in algorithms is a core competency we want you to acquire in this course.

Reminder Objects

```
class Car {  
    String color;  
    int speed = 0;  
    int acceleration = 0;  
  
    Car(String color, int acceleration) {  
        this.color = color;  
        this.acceleration = acceleration;  
    }  
  
    void speedUp() {  
        speed += acceleration;  
    }  
    void brake() {  
        if (speed >= acceleration)  
            speed = speed - acceleration;  
    }  
  
    public static void main(String[] args) {  
        Car myPorsche = new Car("black", 10);  
        myPorsche.speedUp();  
        Car myFerrari = new Car("red", 20);  
        myFerrari.speedUp();  
        myFerrari.speedUp(); // speed is now 40  
    }  
}
```

Encapsulated instance
variables

Constructor

Methods



In Java, everything is a reference*

- Definition: A **reference** is a variable that refers to something else and can be used as an alias for that something else.
- No pointer arithmetic possible with reference like with C/C++ pointers. The access is safe-guarded by Java. A pointer is a reference but not the opposite. A reference is a limited pointer.
- * except primitive types: int, boolean, char, etc.

More on references

```
public static void main(String[] args) {  
    Car car1 = new Car("black", 5);  
    Car car2 = car1;  
}
```

car1 is a reference to an actual Car object

The “=” copies the reference and assign it to car2. Thus car2 is a reference to the same object as car1 but there is no object creation here

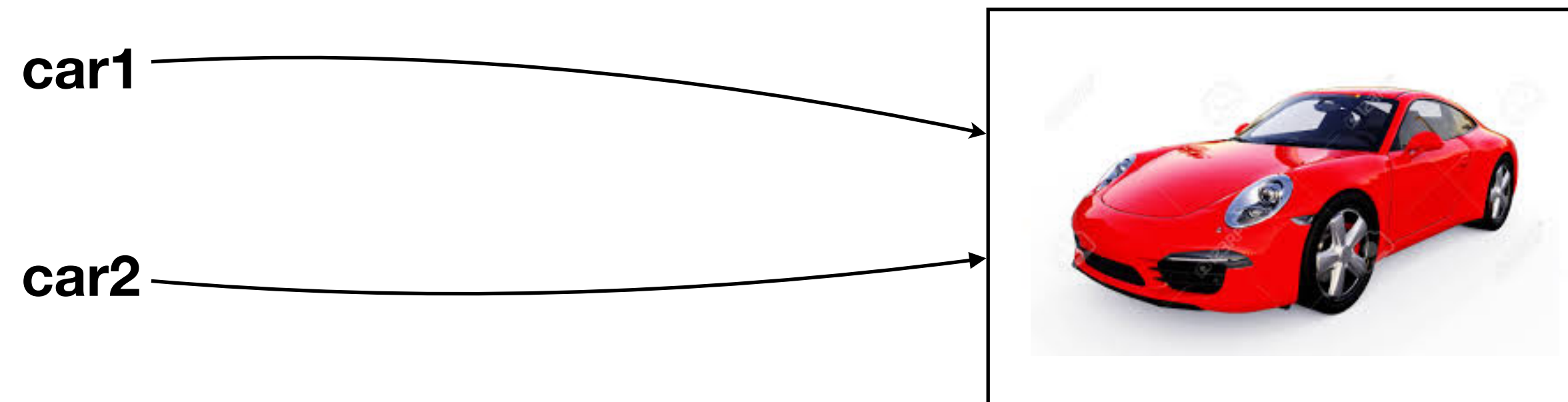
car1

car2



More on references

```
public static void main(String[] args) {  
    Car car1 = new Car("black",5);  
    Car car2 = car1;  
    car2.color = "red";  
}
```



Object vs Primitive types

- Primitive types in Java:
 - boolean – 1 bit
 - byte – 8 bits
 - short, char – 16 bits
 - int, float – 32 bits
 - long, double – 64 bits
- Java also has the object representation of Primitive types:
 - Boolean – 128 bits
 - Byte – 128 bits
 - Short, Character – 128 bits
 - Integer, Float – 128 bits
 - Long, Double – 192 bits
- autoboxing and unboxing = From primitive types to objects and vice versa

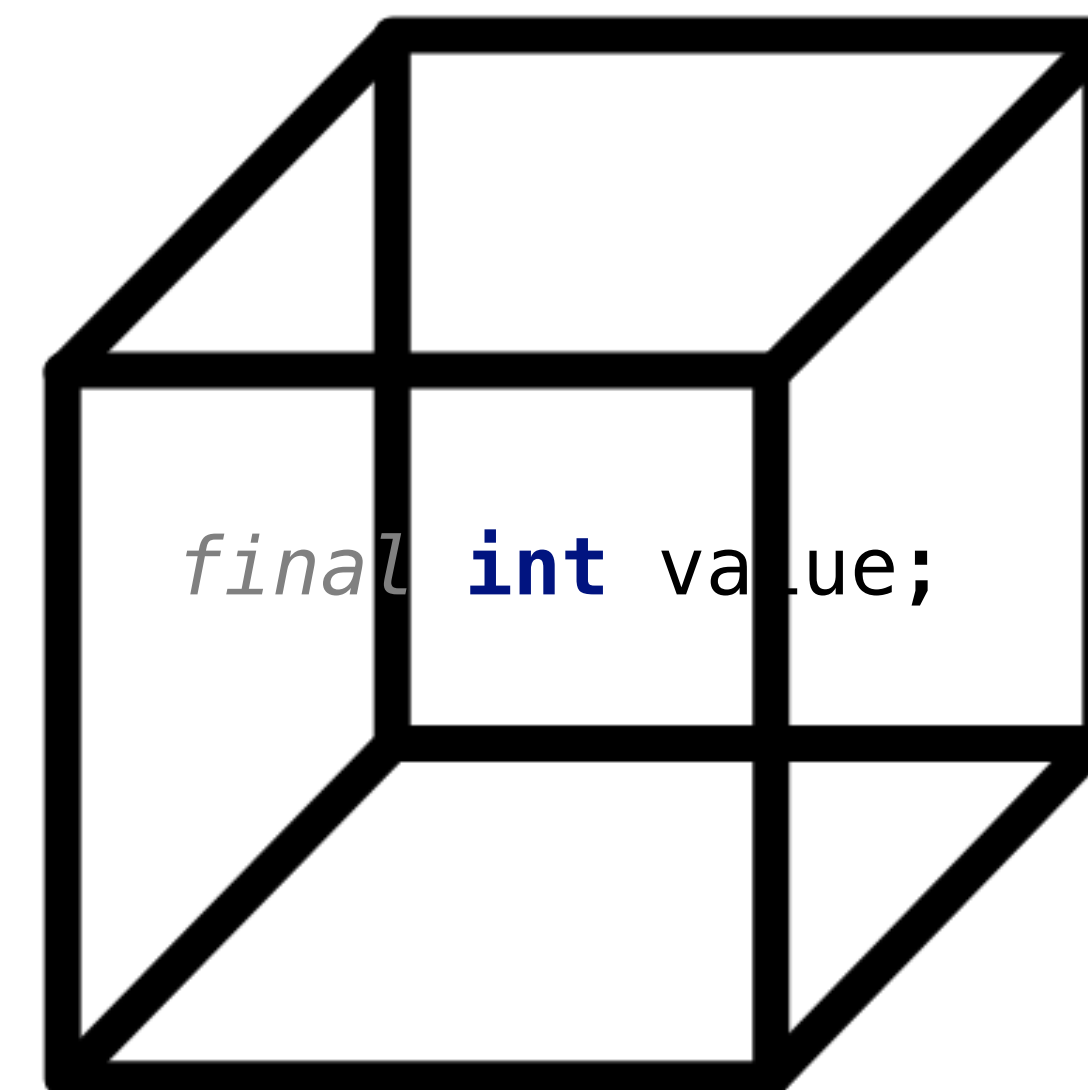
Auto and out boxing

Every object contains a single value of the corresponding primitive type. The **wrapper classes are immutable** (so that their state can't change once the object is constructed) and are final (so that we can't inherit from them).

Under the hood, Java performs a conversion between the primitive and reference types if an actual type is different from the declared one. This conversion is called **auto-boxing** and **out-boxing**

```
public static void main(String[] args) {  
    Integer n1 = 3; // auto-boxing  
    int n2 = n1; // out-boxing  
    foo(n2); // auto-boxing  
}  
  
public static int foo(Integer n1) {  
    return n1+1;  
}
```

The integer class is just
a wrapper around an
immutable int field



Exercise

- What is printed ?

```
public static void main(String[] args) {  
    Integer n1 = 3;  
    Integer n2 = n1;  
    n2 = 4;  
    System.out.println(n1);  
}
```

Parameters are passed by value

- Java manipulate objects by reference but arguments are are not passed by reference but **by value**.
- What is printed here ?

```
public static void main(String[] args) {  
    int n1 = 1;  
    int n2 = 2;  
    swap(n1,n2);  
    System.out.println(n1+" "+n2);  
}  
  
public static void swap(int n1, int n2) {  
    int tmp = n1;  
    n1 = n2;  
    n2 = tmp;  
}
```

By value means that
the value is copied

By value for Objects ?

- Java manipulate objects by reference but arguments are are not passed by reference but **by value**.
- What is printed here ?

```
public static void main(String[] args) {  
    int n1 = 1;  
    int n2 = 2;  
    swap(n1,n2);  
    System.out.println(n1+" "+n2);  
}
```

```
public static void swap(int n1, int n2) {  
    int tmp = n1;  
    n1 = n2;  
    n2 = tmp;  
}
```

By value means that the values n1 and n2 are copied (fresh variables).

By value for object (references)?

```
public static void main(String[] args) {  
    Car myPorsche;  
    myPorsche = new Car("black",5);  
  
    tuneIntoAFerrari(myPorsche);  
  
    // myPorsche is still a reference to the black  
    System.out.println(myPorsche);  
}
```

myPorsche



c

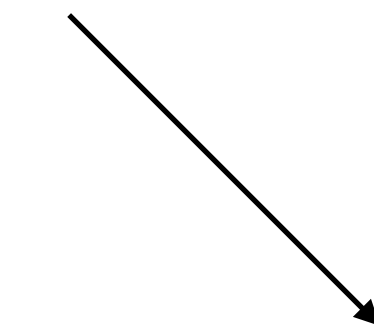
```
public static void tuneIntoAFerrari(Car c) {  
    c = new Car("red",10); // c is now a reference to a new "red" car  
}
```



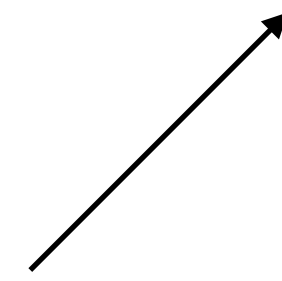
By value for Objects ?

```
public static void main(String[] args) {  
    Car myPorsche = new Car("black",5);  
    tuneIntoAFerrari(myPorsche);  
}  
  
public static void tuneIntoAFerrari(Car c) {  
    c.color = "red";  
    c.acceleration = 10;  
}
```

myPorsche



c



The null reference

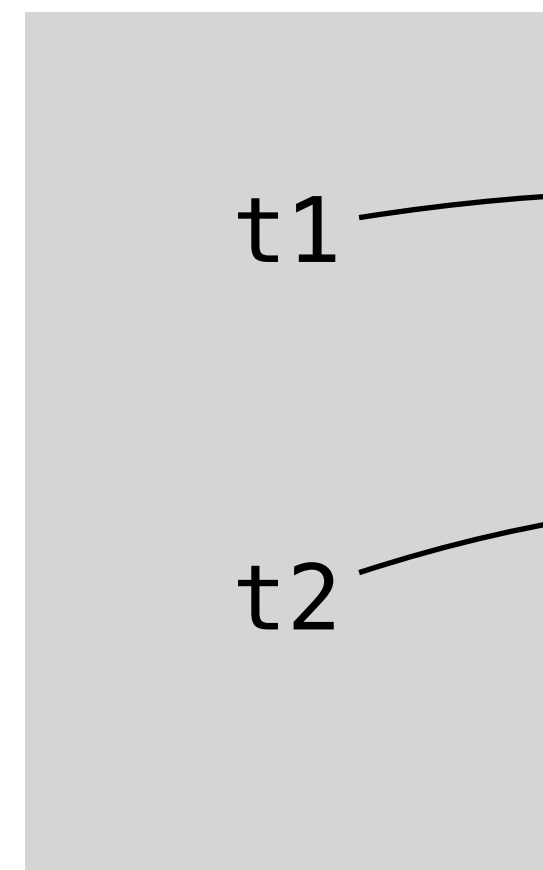
- A uninitialized reference is ***null***

```
public static void main(String[] args) {  
    Car myPorsche;  
    System.out.println(myPorsche);  
}
```

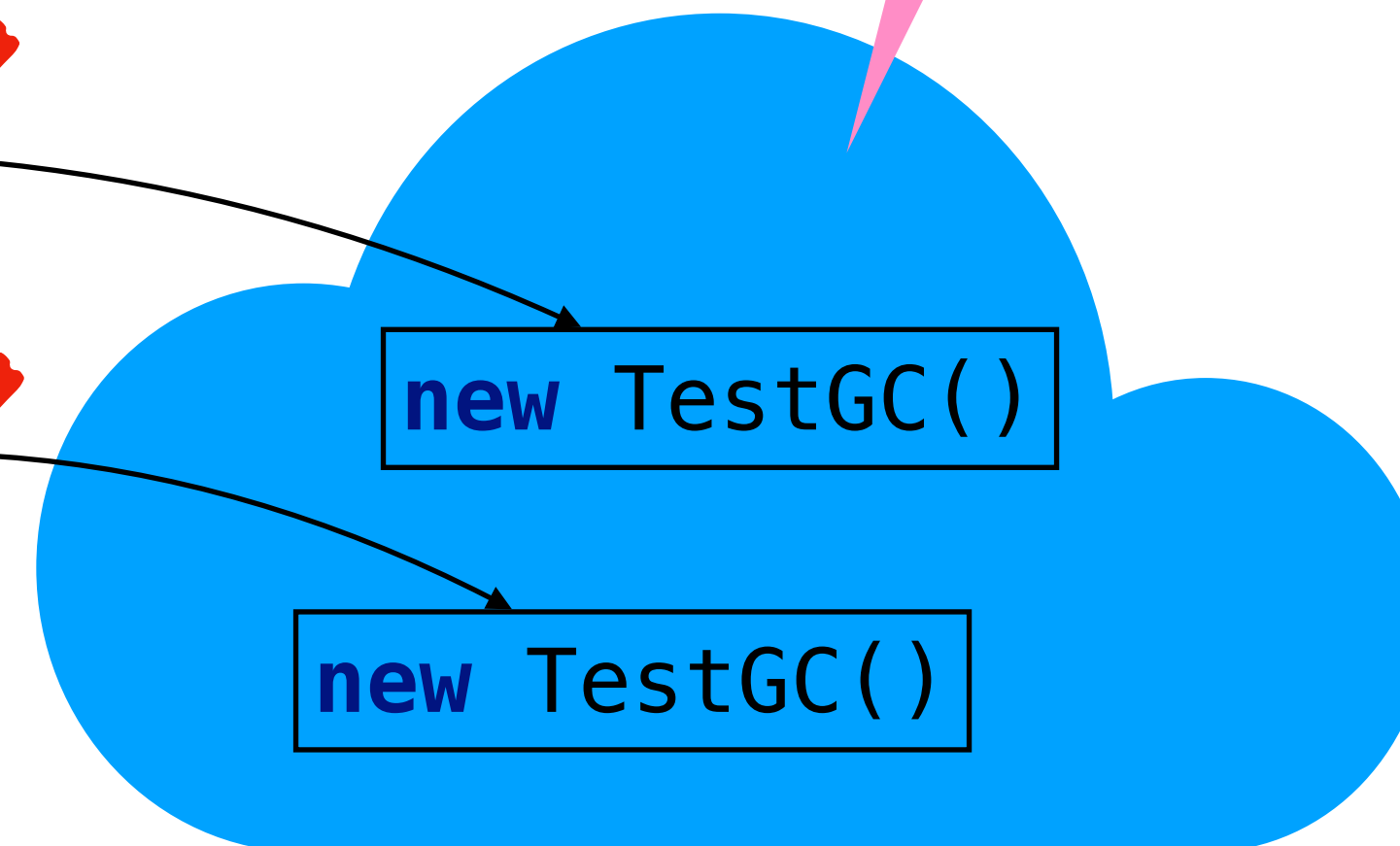
Garbage Collection Revisited

```
public class TestGC {  
    public static void main(String[] args) {  
        TestGC t1 = new TestGC();  
        TestGC t2 = new TestGC();  
  
        // Nullifying the reference variable  
        t1 = null;  
  
        // Nullifying the reference variable  
        t2 = null;  
    }  
}
```

Stack Frame



Heap

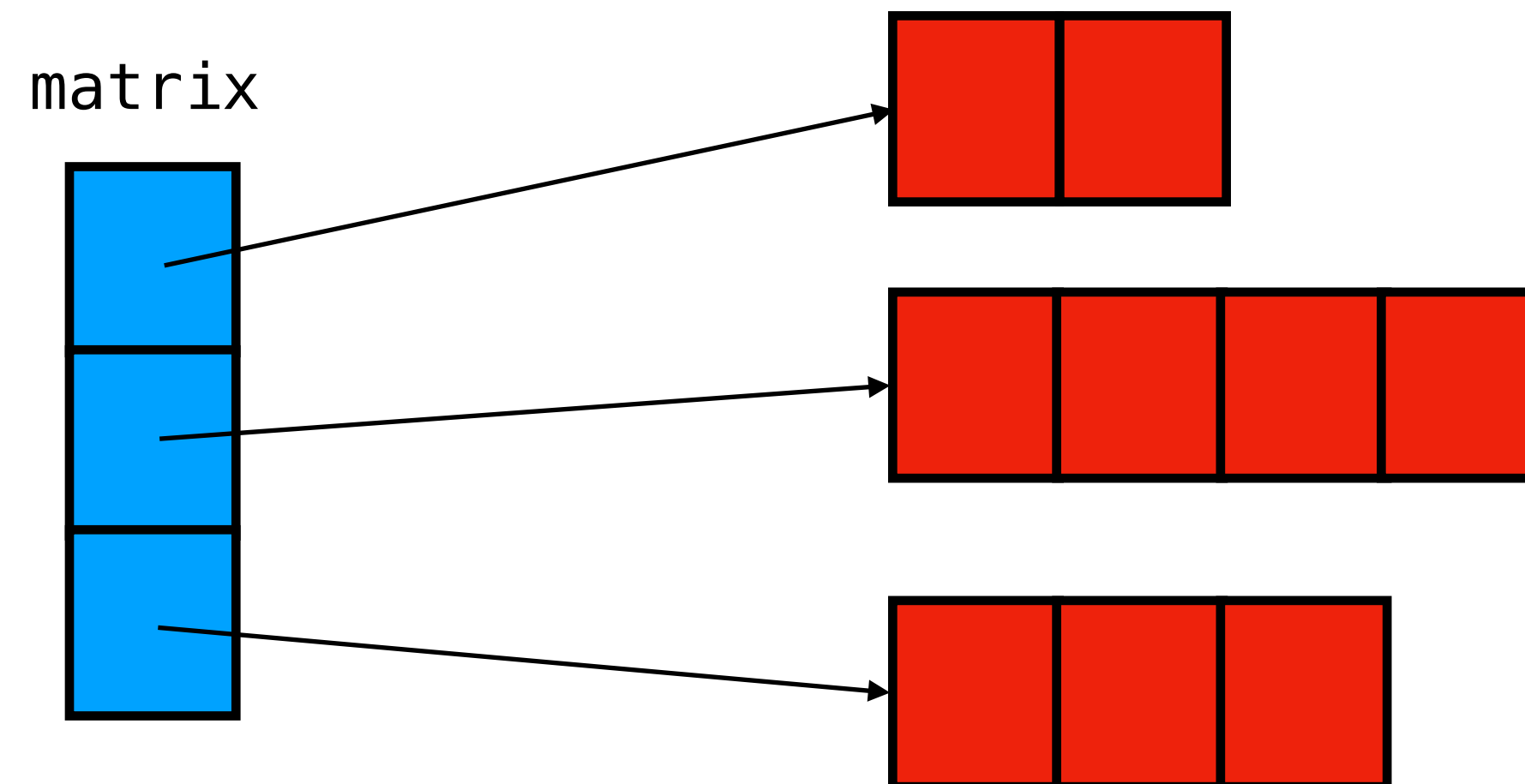


The two instances
are not accessible
any more, they can
be GC

Java Arrays of Arrays (Rappel)

```
public class ArrayDemo {  
    public static void main(String[] args) {  
        int[][] matrix; // declaration of an array of array of int's  
        // declare an array of three references to int [] arrays  
        matrix = new int[3][];  
        // initialize each array individually  
        matrix[0] = new int [] { 1, 0};  
        matrix[1] = new int []{ 1, 0, 12, -1};  
        matrix[2] = new int []{-5, -2, 2};  
    }  
}
```

Pas nécessairement
la même taille



Iterating over an array (with indices)

```
int[][] matrix; // declaration

matrix = new int[][]{ { 1,  0, 12, -1},
                      { 7, -3,  2,  5},
                      {-5, -2,  2, -9}
};

matrix = new int[3][];
matrix[0] = new int [] { 1,  0, 12, -1};
matrix[1] = new int []{-5, -2,  2, -9};
matrix[2] = new int []{ 1,  0, 12, -1};

for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        System.out.println("entry " + i + "," + j + "=" + matrix[i][j]);
    }
}
```

Iterating on values

```
int[][] matrix; // declaration

matrix = new int[][]{{1, 0, 12, -1},
                    {7, -3, 2, 5},
                    {-5, -2, 2, -9}
};

matrix = new int[3][];
// initialize each array
matrix[0] = new int[]{1, 0, 12, -1};
matrix[1] = new int[]{1, 0, 12, -1};
matrix[2] = new int[-5, -2, 2, -9];

for (int[] line : matrix) {
    for (int v : line) {
        System.out.println(v);
    }
}
```

Java Exercise

```
public class Main {  
  
    public static void main(String[] args) {  
        int [] a = new int[10];  
        foo(a);  
        System.out.println(Arrays.toString(a));  
    }  
  
    public static void foo(int [] a) {  
        a[0] = 2;  
    }  
}
```



- What is the output ?
 - [2, 0, 0, 0, 0, 0, 0, 0, 0, 0]
 - [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
 - NullPointerException

Java Exercise

```
public class Main {  
  
    public static void main(String[] args) {  
        int a = 7;  
        foo(a);  
        System.out.println(a);  
    }  
  
    public static void foo(int a) {  
        a = 6;  
    }  
}
```



- What is the output ?
 - 6
 - 7
 - Other

Java Exercise

```
public class Main {  
  
    public static void main(String[] args) {  
        String [] a = new String[5];  
        System.out.println(a[0].isEmpty());  
    }  
}
```



- What is the output
 - true
 - false
 - NullPointerException

Reference testing

```
public static void main(String[] args) {  
  
    Integer i1;  
    Integer i2;  
  
    i1 = 160;  
    i2 = 160;  
  
    System.out.println(i1 == i2); // false  
  
    String s1 = "LSINF1102";  
  
    String s3 = s1+"";  
  
    System.out.println(s1 == s3); // false  
  
    Car c1 = new Car("black",5);  
    Car c2 = new Car("black",5);  
  
    System.out.println(c1 == c2); // false  
}
```

For (logical equality testing) use equals

```
public static void main(String[] args) {  
  
    Integer i1;  
    Integer i2;  
  
    i1 = 160;  
    i2 = 160;  
  
    System.out.println(i1.equals(i2)); // true  
  
    String s1 = "LSINF1102";  
  
    String s3 = s1+"";  
  
    System.out.println(s1.equals(s3)); // true  
  
    Car c1 = new Car("black",5);  
    Car c2 = new Car("black",5);  
  
    System.out.println(c1.equals(c2)); // false (by default same behaviour as ==)  
}
```

User Defined Equals

```
class Car {  
  
    String color;  
    int speed = 0;  
    int acceleration = 0;  
  
    Car(String color, int acceleration) {  
        this.color = color;  
        this.acceleration = acceleration;  
    }  
  
    @Override  
    public boolean equals(Object obj) {  
        if (obj instanceof Car) {  
            Car c1 = (Car) obj;  
            return color.equals(c1.color) && speed == c1.speed;  
        }  
        return false;  
    }  
  
    public static void main(String[] args) {  
  
        Car c1 = new Car("black",5);  
        Car c2 = new Car("black",5);  
  
        System.out.println(c1.equals(c2)); // true  
    }  
}
```

Some strange behaviour's (source of bug 🤯)

```
public static void main(String[] args) {  
    int v1 = 2;  
    int v2 = 2;  
  
    System.out.println(v1 == v2); // true  
  
    Integer i1 = 2;  
    Integer i2 = 2;  
  
    System.out.println(i1 == i2); // true (because of caching)  
  
    i1 = 160;  
    i2 = 160;  
  
    System.out.println(i1 == i2); // false  
  
    String s1 = "LSINF1102";  
    String s2 = "LSINF1102";  
  
    System.out.println(s1 == s2); // true because of string constant "interning"  
  
    String s3 = s1+"";  
  
    System.out.println(s1 == s3); // false  
}
```

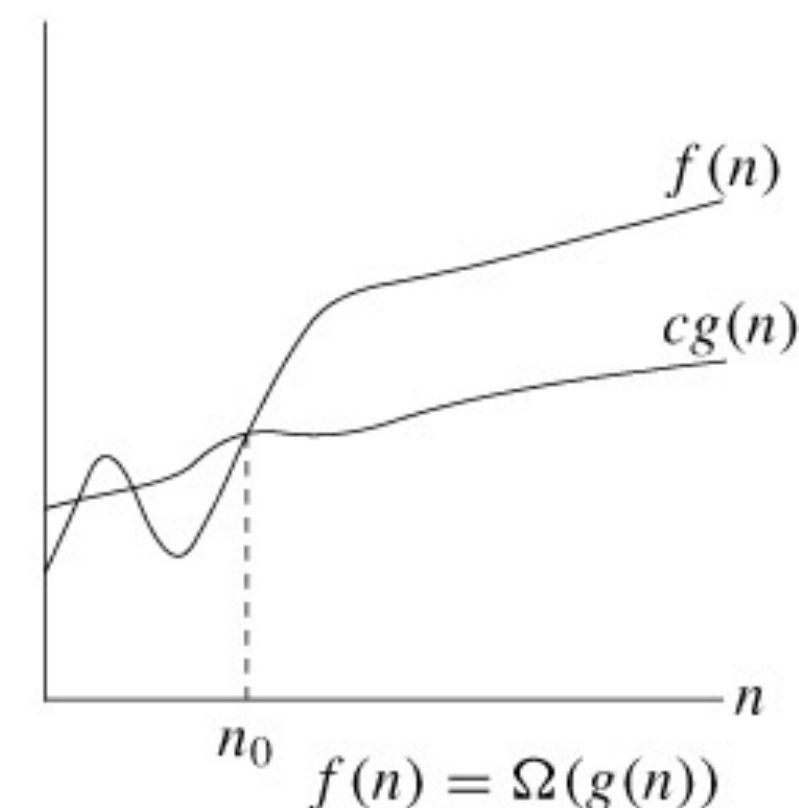
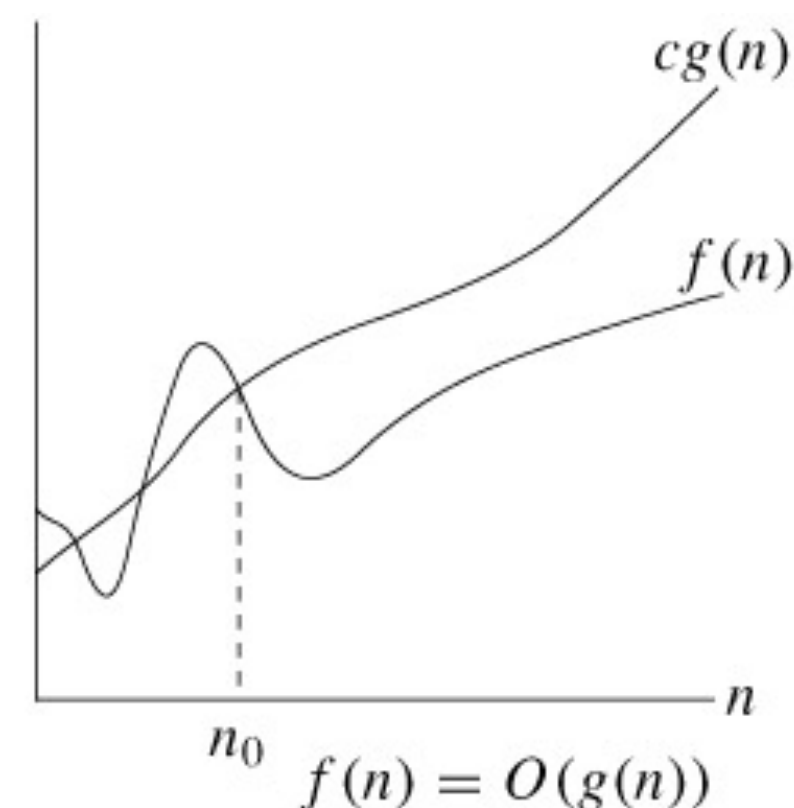
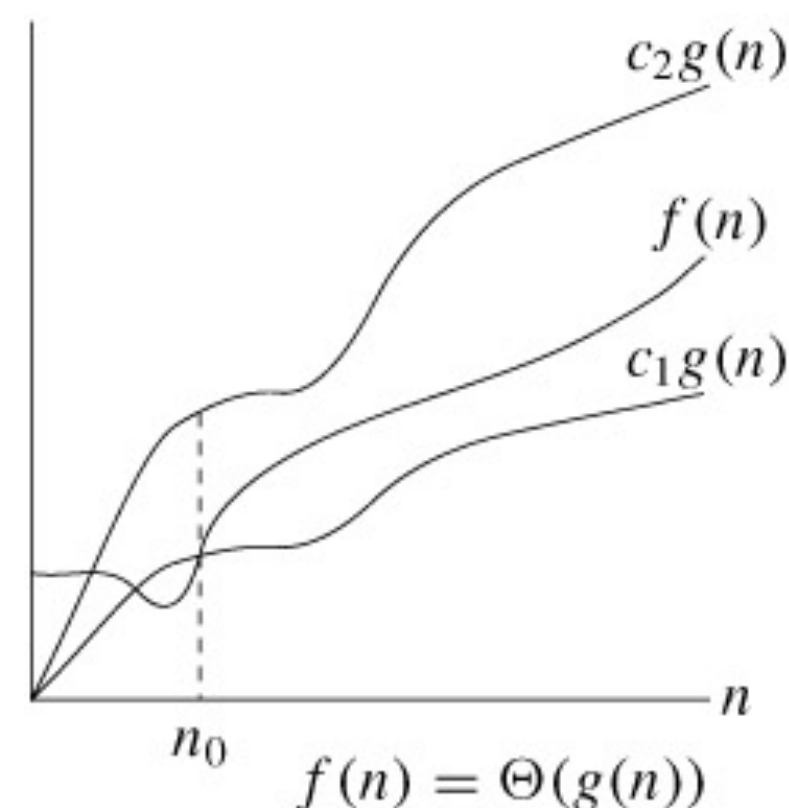
La complexité des algorithmes

- D.E. Knuth posited that it is possible to build a mathematical model to describe the running time of algorithms. Indeed, the running time relies on two main factors:
 - The cost of executing each instruction (difficult to estimate, depends on the JVM, the OS).
 - The frequency of execution of each instruction (can be calculated because it depends solely on the algorithm and the input data).



Complexities

- $T(n)$ pour $n \in \mathbb{N} = \{0, 1, 2, \dots\}$
 - est une fonction du temps de calcul d'un algorithme pour une entrée de taille n
- Etant donné une fonction $g(n)$ pour $n \in \mathbb{N} = \{0, 1, 2, \dots\}$
 - $\Theta(g(n)) = \{ f(n) : \text{il existe les constantes positives } c_1, c_2 \text{ et } n_0 \text{ tq } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0 \}$
 - $O(g(n)) = \{ f(n) : \text{il existe les constantes } c \text{ et } n_0 \text{ tq } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0 \}$
 - $\Omega(g(n)) = \{ f(n) : \text{il existe les constantes } c \text{ et } n_0 \text{ tq } 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0 \}$
- On devrait écrire $T(n) \in \Theta(g(n))$ mais on écrit généralement $T(n) = \Theta(g(n))$



La notation « $\sim$ » (order of growth function)

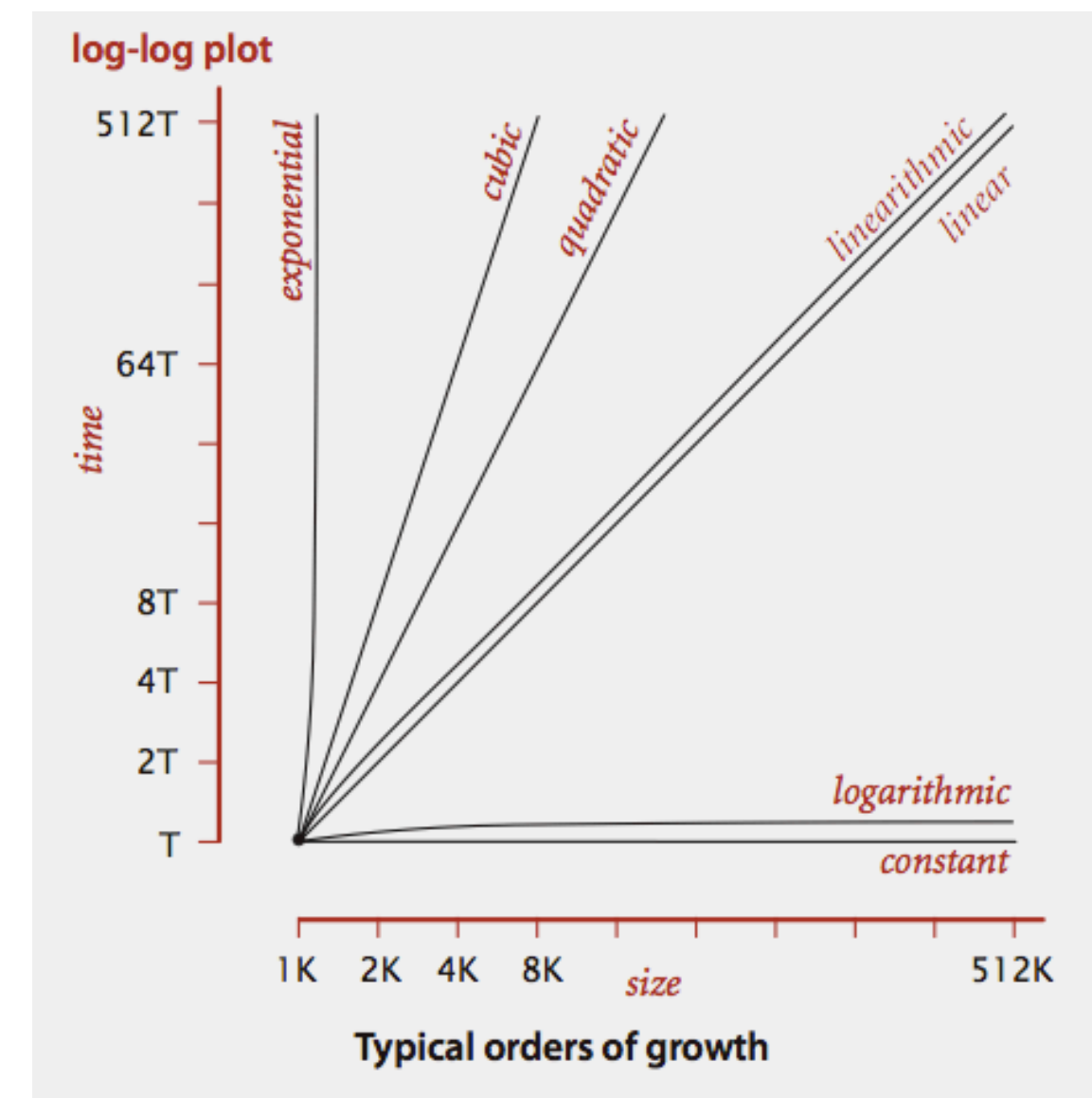
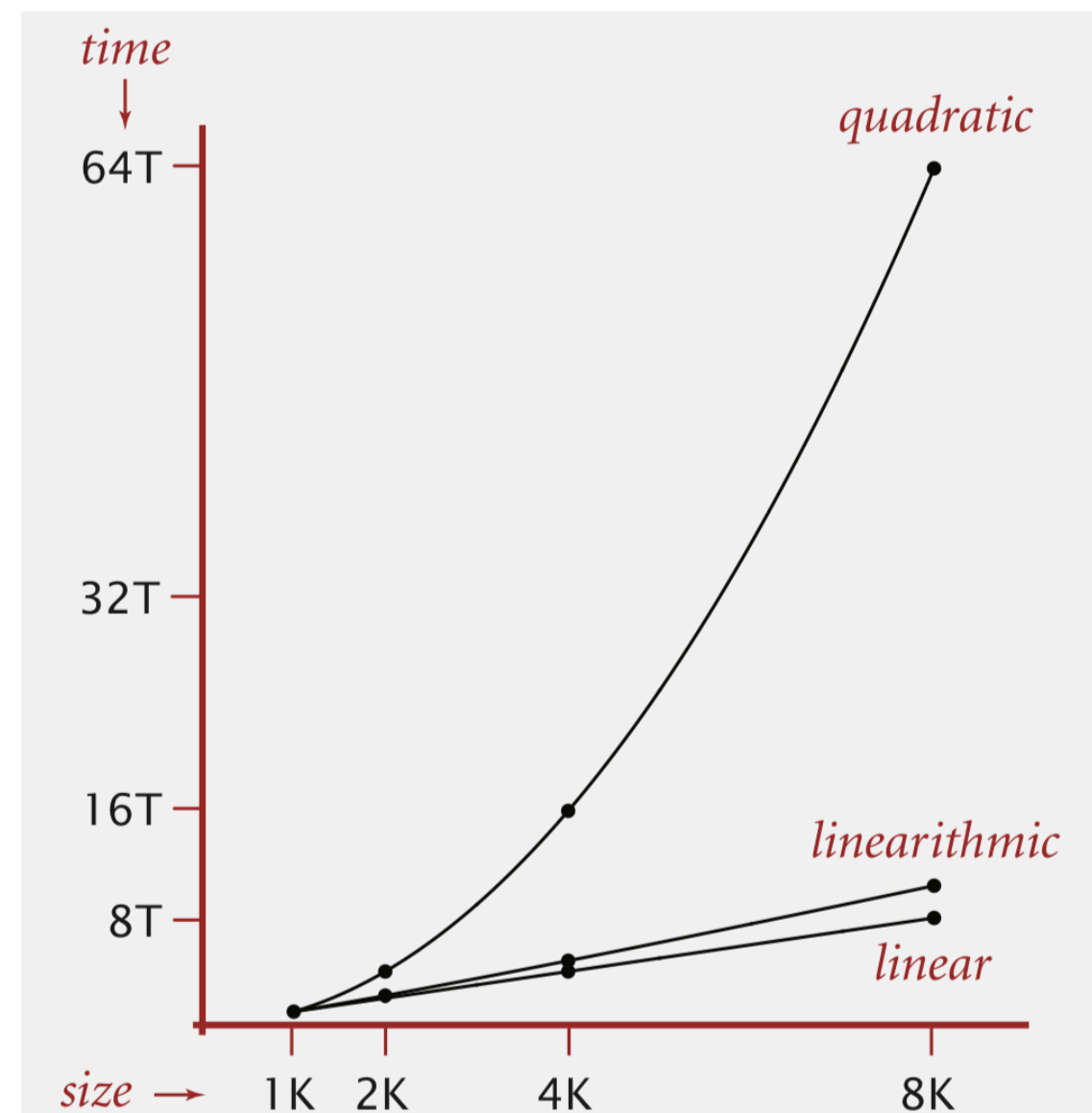
- The book uses a different, less standard definition of complexity:

$$\sim g(n) = \left\{ f(n) : \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 1 \right\}$$

- We write $f(n) \sim g(n)$ to express that $\frac{f(n)}{g(n)}$ approaches 1 as n increases—that is, for large values of n , f and g behave in the same way.
- For most algorithms, $f(n)$ has the form $a \cdot g(n)$ with $g(n) = n^b (\log(n))^c$, referred to as the '**order of growth function**'.

En pratique

- You can consider that $O(n^2)$ becomes impractical quite quickly for $n > 10^6$



	insertion sort $O(n^2)$			merge sort $O(n \log(n))$		
taille	10^3	10^6	10^9	10^3	10^6	10^9
temps	instantané	2.8 h	317 ans	instantané	1 s	18 min

Example 3SUM algorithm

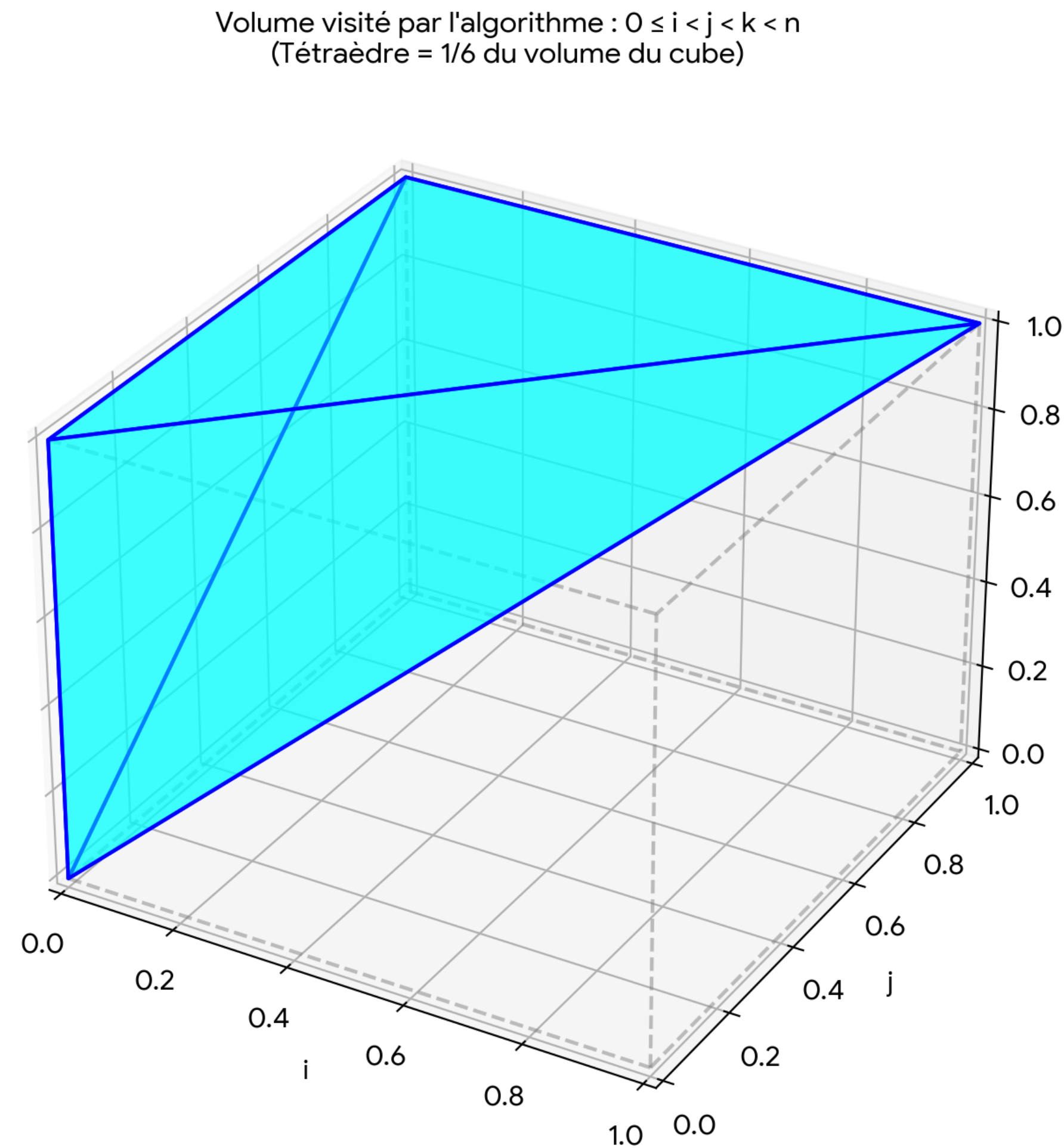
Count number of distinct triplets with sum equal to 0

```
public static int count(int[] a) {  
    int n = a.length;  
    int count = 0;  
    for (int i = 0; i < n; i++) {  
        for (int j = i+1; j < n; j++) {  
            for (int k = j+1; k < n; k++) {  
                if (a[i] + a[j] + a[k] == 0) {  
                    count++;  
                }  
            }  
        }  
    }  
    return count;  
}
```

Time complexity ?

Complexity $\Theta(n^3)$ or $\sim (n^3/6)$

- If we normalize the indices $[0,1]^3$, the fraction of the 3D array visited is



Does an algorithm exist for computing the time complexity?

- Could we conceive an algo able to compute the time complexity of count without analyzing the source code, just by running it for some inputs (blackbox) ...
 - `public static int count(int[] a)`
- ... The only thing we can do is use `System.currentTimeMillis()` for different inputs ?
- Yes, it is called the « ***doubling ratio test*** »



Doubling ratio test*

Si $T(N) \sim a N^b \log(N)$ alors $T(2N)/T(N) \sim 2^b$

- Proof

$$\bullet \frac{T(2N)}{T(N)} = \frac{a(2N)^b \log(2N)}{aN^b \log(N)} = 2^b \frac{\log(2) + \log(N)}{\log(N)} = 2^b \left(1 + \frac{\log(2)}{\log(N)}\right) \sim 2^b$$

- If we measure $T(2N)/T(N)$ for a large N and we take the log, we get b .
- Example: 3SUM: The ratio tends to 8 thus $b = \log(8) = 3$

N	time	ratio
1000	0.1	
2000	0.8	8
4000	6.4	8
8000	51.1	7.98

* log is base 2

Exam Question 2019

- We have measured the computation time for some input sizes (N) of a algorithm

N	time (s)
100	5
200	40
400	320
800	2560

- What is the « order of growth » function for this algorithm ?

Java Exercise

```
public static void foo(int n) {  
    int [] a = new int[n];  
}
```



- What is the time complexité $T(n)$ of foo methods where n is the value passed in argument ?
 - $\Theta(n)$
 - $\Theta(1)$
 - Other

Java Exercise

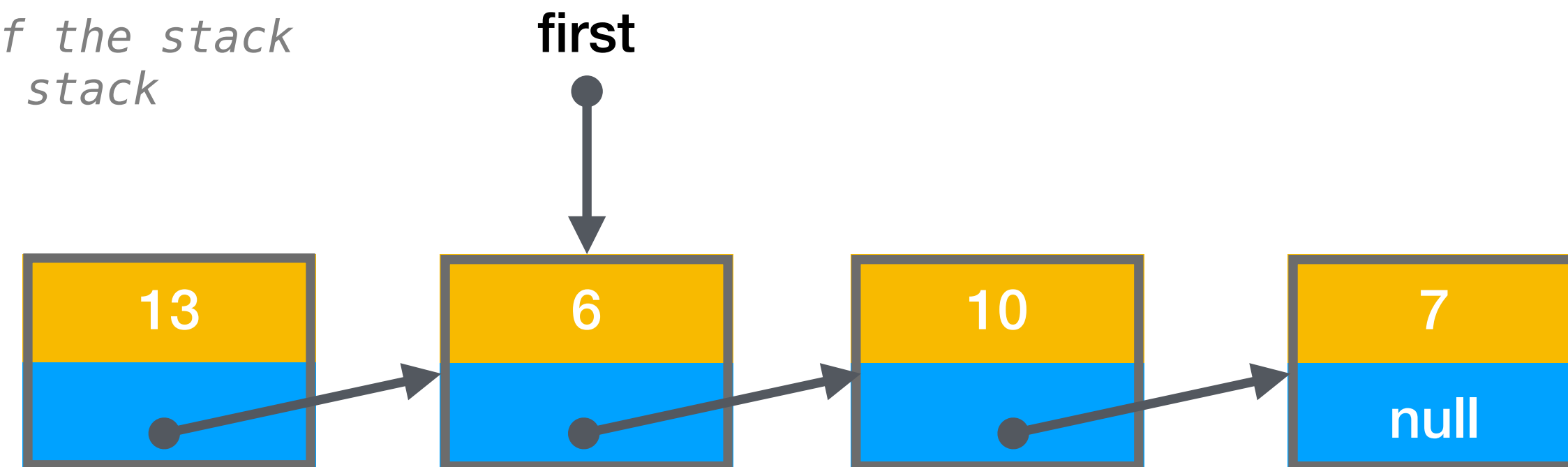
```
public static int foo(int [] a) {  
    return a.length;  
}
```



- What is the time complexity $T(n)$ of the foo **method** where n is the size of the array ?
 - $\Theta(n)$
 - $\Theta(1)$
 - Other

Stack with a LinkedList

```
public class LinkedStack<Item> {  
  
    private int n;           // size of the stack  
    private Node first;      // top of stack  
  
    // helper linked list class  
    private class Node {  
        private Item item;  
        private Node next;  
    }  
  
    public LinkedStack() {  
        first = null;  
        n = 0;  
        assert check();  
    }  
  
    public boolean isEmpty() { return first == null; }  
    public int size() { return n; }  
    public void push(Item item) {  
        Node oldfirst = first;  
        first = new Node();  
        first.item = item;  
        first.next = oldfirst;  
        n++;  
    }  
  
    public Item pop() {  
        if (isEmpty()) throw new NoSuchElementException("Stack underflow");  
        Item item = first.item;           // save item to return  
        first = first.next;               // delete first node  
        n--;  
        return item;                       // return the saved item  
    }  
}
```



push(13)

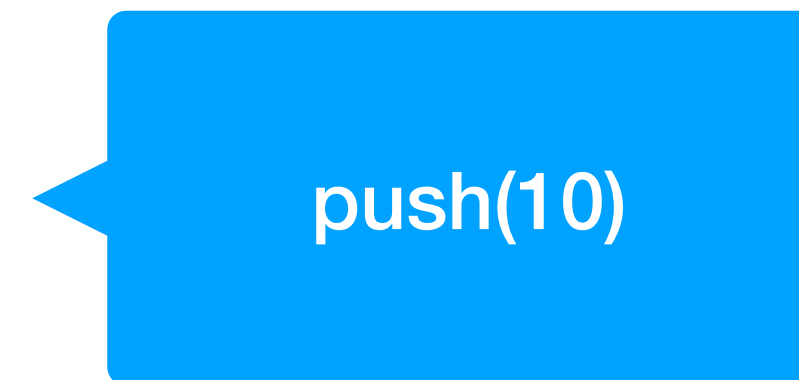
Complexité pour le
push/pop?

Stack with an Array

- Insertion: if the end of the array is reached, double the array size and copy all elements into it.

0	1	2	3	4	5	6	7
5	10	5	9	10	null	null	null

- $n=5$



Stack with an Array

```
public class ResizingArrayStack<Item> {
    private Item[] a;           // array of items
    private int n;              // number of elements on stack

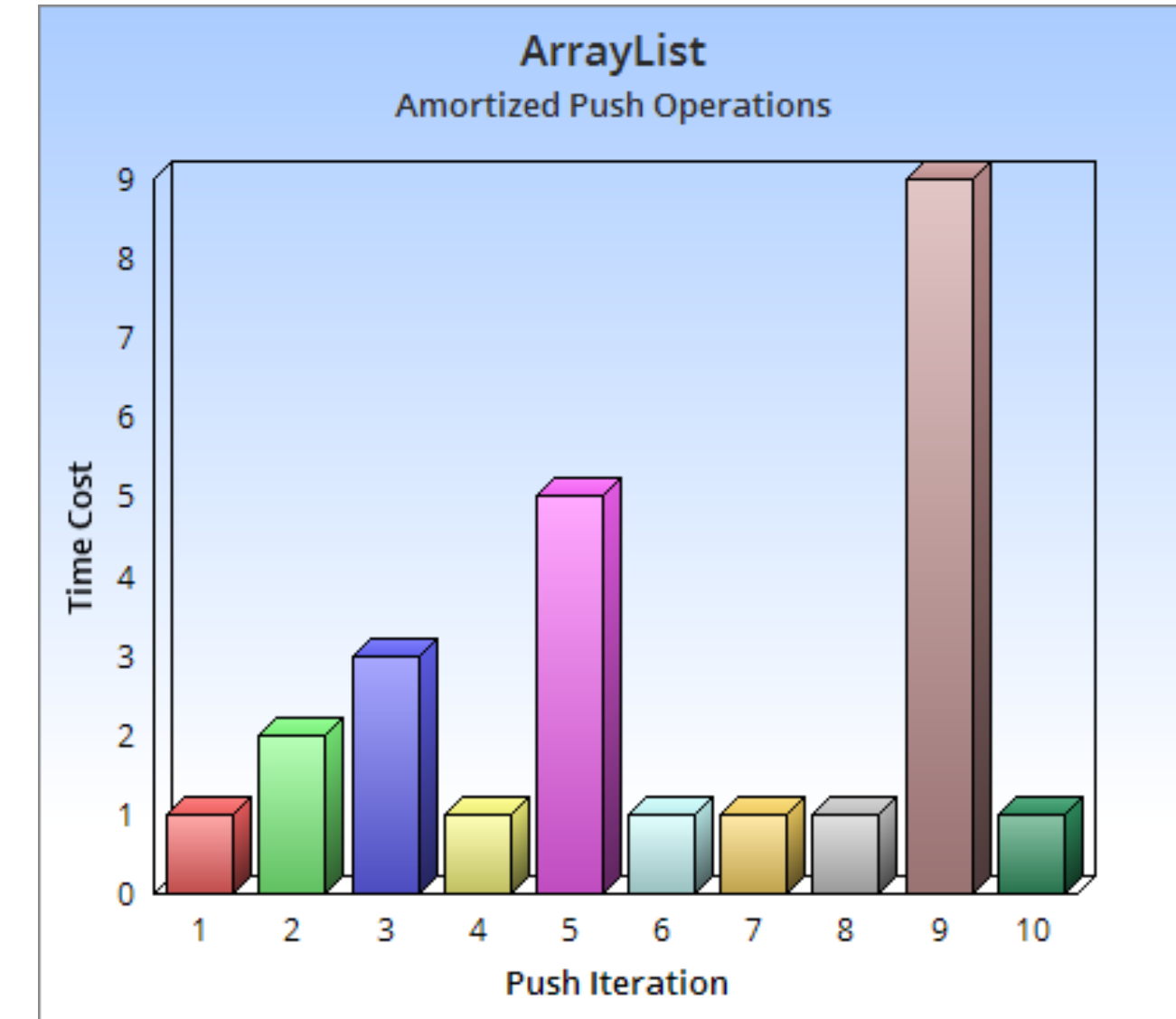
    public ResizingArrayStack() {
        a = (Item[]) new Object[2];
        n = 0;
    }
    public boolean isEmpty() { return n == 0; }
    public int size() { return n; }
    private void resize(int capacity) {
        a = java.util.Arrays.copyOf(a, capacity);
    }
    public void push(Item item) {
        if (n == a.length) resize(2 * a.length); // 2*size if necessary
        a[n++] = item;                             // add item
    }
    public Item pop() {
        if (isEmpty()) throw new NoSuchElementException("Stack underflow");
        Item item = a[n - 1];
        a[n - 1] = null; // Important for Garbage Collection
        n--;
        // shrink size of array if necessary
        if (n > 0 && n == a.length / 4) resize(a.length / 2);
        return item;
    }
}
```

```
Item[] temp = (Item[]) new Object[capacity];
for (int i = 0; i < n; i++) {
    temp[i] = a[i];
}
a = temp;
```

Complexité du
push/pop?

Amortized Time Complexity

- Cost of a push() Operation:
 - Takes $O(1)$ if no resizing is required.
 - Takes $\Theta(n)$ when the array must double in capacity.
- Amortized Analysis:
 - Performing $n + 1$ consecutive push() operations starting with an array of capacity n .
 - Average cost per operation of $O(1)$ **amortized** :
$$\frac{O(1) \cdot n + O(n)}{n + 1} = O(1)$$
- Important Distinction:
 - The complexity of an arbitrary single push() is $\Omega(1)$ (best case) and $O(n)$ (worst case), where n is the number of elements in the stack.



Expected time complexity

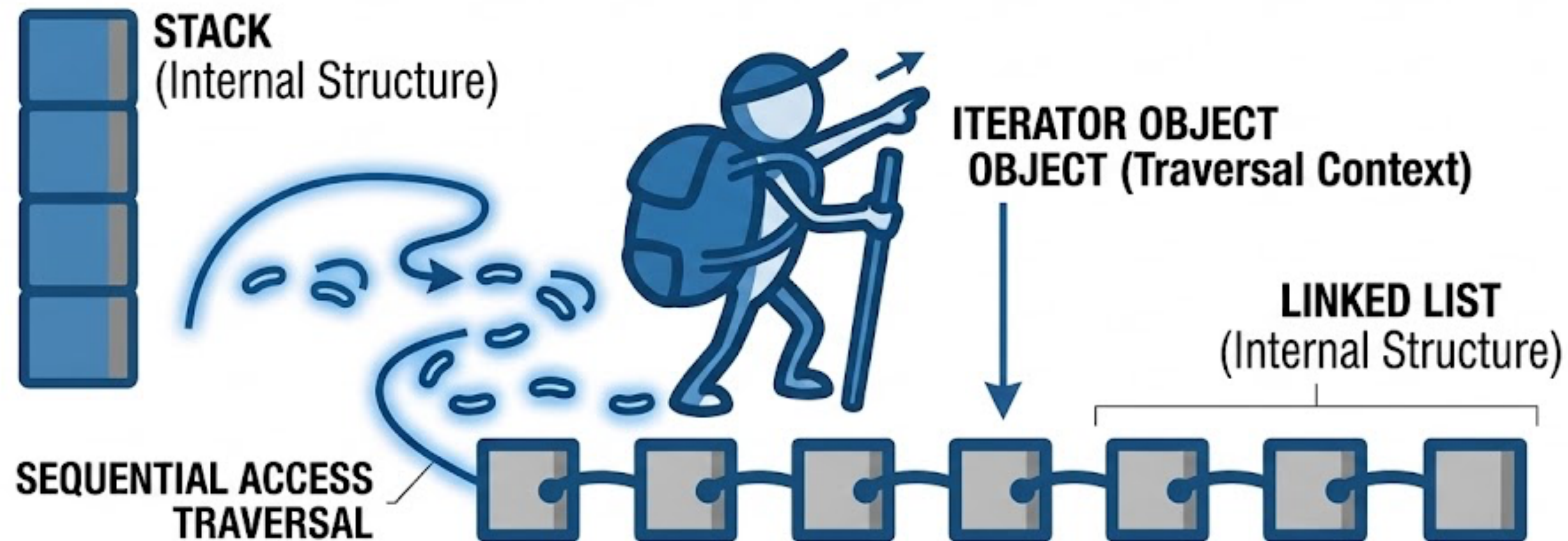
- Data-Dependent Complexity: For some algorithms, time complexity depends on the actual values/order of the input, not just the input size n .
- Example — Quicksort:
 - $O(n^2)$ in the worst case (e.g., when values are already sorted using a naive pivot).
 - $O(n \log n)$ when input values are initially randomly ordered.
- Mitigating the Worst Case: Randomly shuffling the input prior to sorting prevents the adversarial worst-case scenario.
- Expected Time Complexity: This yields an expected time complexity of $O(n \log n)$

Iterator Pattern

- Iterator: Provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation.

ITERATOR PATTERN: COLLECTION TRAVERSAL

Abstracts the logic of collection traversal into a separate object (a traversal context) to iterate over any collection type (e.g., Stack or Linked List) without exposing its implementation.



Iterator Interface

```
public interface Iterator<E> {  
    /**  
     * Returns {@code true} if the iteration has more elements.  
     * (In other words, returns {@code true} if {@link #next} would  
     * return an element rather than throwing an exception.)  
     *  
     * @return {@code true} if the iteration has more elements  
     */  
    boolean hasNext();  
    /**  
     * Returns the next element in the iteration.  
     *  
     * @return the next element in the iteration  
     * @throws NoSuchElementException if the iteration has no more elements  
     */  
    E next();  
    /**  
     * Removes from the underlying collection the last element returned  
     * by this iterator (optional operation). This method can be called  
     * only once per call to {@link #next}.  
     * 

The behavior of an iterator is unspecified if the underlying collection  
     * is modified while the iteration is in progress in any way other than by  
     * calling this method, unless an overriding class has specified a  
     * concurrent modification policy.

  
     *   
     */  
    default void remove() {  
        throw new UnsupportedOperationException("remove");  
    }  
}
```

My advise is not to implement the “remove” because it is optional and complicates a lot the code of an iterator

Iterator Usage

```
List<Integer> list = new LinkedList<>();  
list.add(1);  
list.add(2);  
list.add(3);  
  
Iterator<Integer> iterator = list.iterator();  
while (iterator.hasNext()) {  
    System.out.println(iterator.next());  
}
```

Iterable Interface

```
public interface Iterable<T> {  
    /**  
     * Returns an iterator over elements of type {@code T}.  
     *  
     * @return an Iterator.  
     */  
    Iterator<T> iterator();  
  
}
```

Iterable and for-loops

```
List<Integer> list = new LinkedList<>();  
list.add(1);  
list.add(2);  
list.add(3);  
  
for (Integer i : list) {  
    System.out.println(i);  
}
```



```
Iterator<Integer> iterator = list.iterator();  
while (iterator.hasNext()) {  
    System.out.println(iterator.next());  
}
```

Structure

```
public class LinkedStack<T> implements Iterable<T> {
    private Node<T> top;
    private int size = 0;

    private static class Node<T> {
        // see prev
    }

    public void push(T item) {
        // see prev
    }
    public T pop() {
        // see prev
    }
    public boolean isEmpty() {
        return top == null;
    }
    public int size() {
        return size;
    }

    @Override
    public Iterator<T> iterator() {
        return new LinkedStackIterator();
    }

    private class LinkedStackIterator implements Iterator<T> {

    }
}
```

Node is a static inner/nested class:

- It doesn't need to exist in its own file Node.java
- The outer-class LinkedBag has access to its private fields (no getter/setter)

The Iterator a non static inner/nested class:

- Its existence it bind to a particular instance of LinkedStack.
- It has access to the instance variables of its attached LinkedStack instance.

LinkedList Iterator

```
public class LinkedStack<T> implements Iterable<T> {
```

```
    private Node<T> top;  
    private int size = 0;
```

```
    private static class Node<T> { // see prev }
```

```
    public void push(T item) { // see prev }
```

```
    public T pop() { // see prev }
```

```
    public boolean isEmpty() {return top == null;}
```

```
    public int size() {return size;}
```

```
    @Override
```

```
    public Iterator<T> iterator() {  
        return new LinkedStackIterator();  
    }
```

```
    private class LinkedStackIterator implements Iterator<T> {
```

```
        private Node<T> current = top;
```

```
        @Override
```

```
        public boolean hasNext() {  
            return current != null;  
        }
```

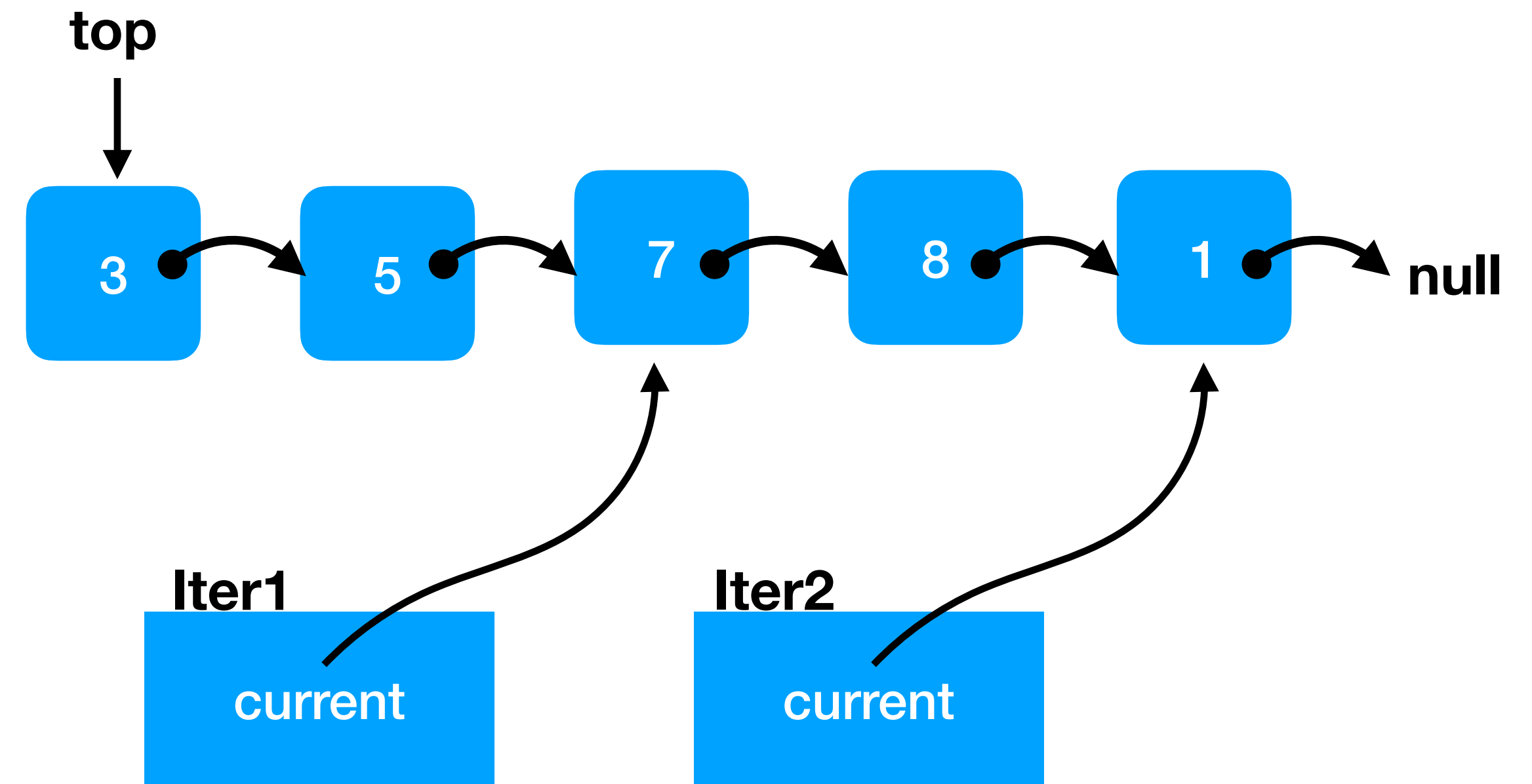
```
        @Override
```

```
        public T next() {  
            if (current == null) throw new IllegalStateException("No more items");  
            T item = current.item;  
            current = current.next;  
            return item;  
        }
```

```
    }
```

```
}
```

```
}
```



Iterator design problem

- Two strategies for implementing a proper iterator:
 - **Fail-Fast:** they throw *ConcurrentModificationException* if there is **structural modification** of the collection.
 - **Fail-Safe:** they don't throw any exceptions if a collection is structurally modified while iterating over it. This is because, they operate on the clone of the collection, not on the original collection and that's why they are called fail-safe iterators.
- Let us fix our iterator to implement the “Fail-Fast” strategy.

```
public static void main(String[] args) {  
    Stack<String> stack = new LinkedStack<String>();  
    stack.push("Computer");  
    stack.push("Table");  
    Iterator ite = stack.iterator();  
    ite.next();  
    ite.next();  
    stack.push("Table");  
    if (ite.hasNext()) {  
        System.out.println("how come you have some next !?!?");  
    }  
}
```

FailFast iterator for LinkedStack

```
public class LinkedStack<T> implements Iterable<T> {
    private Node<T> top;
    private int size = 0;
    private int modCount = 0; // Modification count

    public void push(T item) {
        Node<T> oldTop = top;
        top = new Node<>(item, oldTop);
        size++;
        modCount++;
    }

    public T pop() {
        if (top == null) throw new IllegalStateException("Stack is empty");
        T item = top.item;
        top = top.next;
        size--;
        modCount++;
        return item;
    }

    @Override
    public Iterator<T> iterator() {
        return new LinkedStackIterator();
    }
}
```

Step 1: introduce modification counter

```

public class LinkedStack<T> implements Iterable<T> {
    private Node<T> top;
    private int size = 0;
    private int modCount = 0; // Modification count

    @Override
    public Iterator<T> iterator() {
        return new LinkedStackIterator();
    }

    private class LinkedStackIterator implements Iterator<T> {
        private Node<T> current = top;
        private final int expectedModCount = modCount;

        @Override
        public boolean hasNext() {
            if (expectedModCount != modCount) {
                throw new ConcurrentModificationException();
            }
            return current != null;
        }

        @Override
        public T next() {
            if (expectedModCount != modCount) {
                throw new ConcurrentModificationException();
            }
            if (current == null) throw new IllegalStateException("No more items");

            T item = current.item;
            current = current.next;
            return item;
        }
    }
}

```

Step 2: snapshot of the counter at the creation of the Iterator

Step 3: verify for modifications between creation and iterator usage by comparing the counter

Step 3: verify for modifications between creation and iterator usage by comparing the counter

Each iterator has its own counter

```
LinkedStack<Integer> stack = new LinkedStack<>();
```

```
stack.push(1);  
stack.push(2);  
stack.push(3);
```

```
// modCount should be 3
```

```
Iterator<Integer> iter1 = stack.iterator(); // expectedModCount = 3
```

```
stack.push(4); // modCount = 4;
```

```
Iterator<Integer> iter2 = stack.iterator(); // expectedModCount = 4
```

```
iter1.hasNext(); // 3 != 4 ✨
```

This week <https://pschhaus.github.io/LINFO1121/>

- If needed: Brush up on Java by writing a simple class.
- Tooling: Familiarize yourself with IntelliJ and make sure you can write/execute a basic JUnit test.
- Readings:
 - Read Chapters 1.1–1.4 (feel free to skip what you already know).
 - Review the refresher on computational complexity.
- Preparation:
 - Prepare the theoretical exercises for the lab session.
 - Complete the two INGINious exercises.

Computer Labs

- Subscribe on Moodle to choose your lab session
 - <https://moodle.uclouvain.be/course/view.php?id=1049>
 - Two times slots (UCLouvain):
 - * Thursday 8h30 (Computer Labs or Teams)
 - * Thursday 14h00 (Computer Labs)

One more thing ...



- Les exercices sont open-source
 - https://github.com/pschaus/algorithms_exercises_students (exercices ingénieux)
 - https://github.com/pschaus/algorithms_exercises (solution des exercices ingénieux)
- N'hésitez pas à contribuer activement à l'amélioration du cours (correction de fautes, tests, proposition d'exercices) via des pull-request
 - <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/creating-a-pull-request>